



AUTONOMOUS
INTELLIGENCE GROUP

LECTURE 3 — INSIDE THE DRIVE

FROM A TWIST TO TURNING WHEELS AND SPINNING ROTORS

Topics: Differential drive · Inside the plugin · Odometry · Four wheels · One wheel · Hardware · Rotors

TODAY

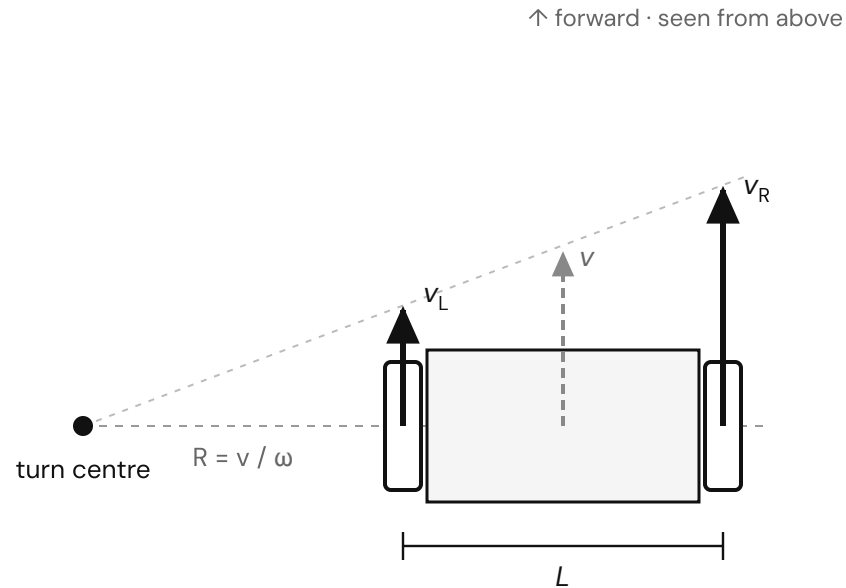
- Differential drive
- Inside the plugin
- Odometry
- Why four wheels lie
- A one-wheel robot
- From simulation to hardware
- **Rotors** — how they differ from a wheel drive

DIFFERENTIAL DRIVE

STEERING WITHOUT A STEERING WHEEL



WHY "DIFFERENTIAL"



$$\omega = \frac{v_R - v_L}{L}$$

- The **difference** between the two wheels *is* the turn
- Equal → straight · unequal → an arc · opposite → a spin on the spot
- No steering wheel: each wheel has **its own motor**. The arrow tips point at the turn centre

LEGEND

v_L, v_R — speed of the left / right tyre over the floor, m/s

L — distance between the two wheels, m

ω — how fast the robot turns, rad/s

v — speed of the robot's centre, m/s

R — distance to the turn centre, m

THE UNICYCLE MODEL

Uni-cycle — a one-wheeled cycle. The model treats the whole robot as **one imaginary wheel**. It rolls where it faces and can turn on the spot. It never slides sideways

$$\dot{x} = v \cos \theta, \quad \dot{y} = v \sin \theta, \quad \dot{\theta} = \omega$$

- **Two** commands move **three** coordinates. Sideways is not a command

$$v = \frac{v_R + v_L}{2}, \quad \omega = \frac{v_R - v_L}{L}$$

- Two real wheels make that one imaginary wheel: the average is v , the difference is ω
- **Two wheels, four wheels, tracks — all are unicycles.** Each side moves as one, and nothing steers. So the robot has only v and ω . Four wheels obey it only approximately: they slide (later)
- A `Twist` uses two of its six numbers: `linear.x` = v , `angular.z` = ω

LEGEND

x, y — where the robot is, m
 θ — where it faces, rad
 $\dot{x}$ — " x per second": a dot is a rate
 v — forward speed, m/s
 ω — turn rate, rad/s
 v_L, v_R — speed of each tyre, m/s
 L — distance between the wheels, m

ONE PLUGIN, ANY NUMBER OF WHEELS

FOUR AND TWO – THE SAME CODE; ONE COMES LATER



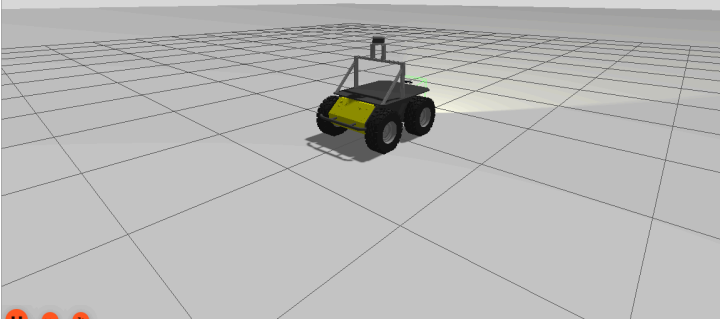
THE PLUGIN BLOCK

```
<!-- Gazebo's stock DiffDrive: the library file and the class inside it -->
<plugin filename="gz-sim-diff-drive-system" name="gz::sim::systems::DiffDrive">
  <left_joint>left_wheel_joint</left_joint>      <!-- a left wheel joint; repeat for more -->
  <right_joint>right_wheel_joint</right_joint>    <!-- a right wheel joint -->
  <wheel_separation>0.35</wheel_separation>      <!-- L, m. Default 1.0 -->
  <wheel_radius>0.1</wheel_radius>               <!-- r, m. Default 0.2 -->
  <topic>/model/bot/cmd_vel</topic>              <!-- IN: gz.msgs.Twist -->
  <odom_topic>/model/bot/odometry</odom_topic>    <!-- OUT: gz.msgs.Odometry -->
</plugin>
```

- Only the joints are required. The defaults of `wheel_separation` and `wheel_radius` are **wrong for every real robot**. It still drives — and its odometry lies
- **In:** `gz.msgs.Twist` — it reads only v and ω . **Out:** `gz.msgs.Odometry` — position, heading, the two speeds. The bridge turns it into `nav_msgs/msg/Odometry`
- Optional tags: publish rate, frame names, limits

 github.com/gazebosim/gz-sim/blob/gz-sim8/src/systems/diff_drive/DiffDrive.hh

FOUR WHEELS – THE HUSKY



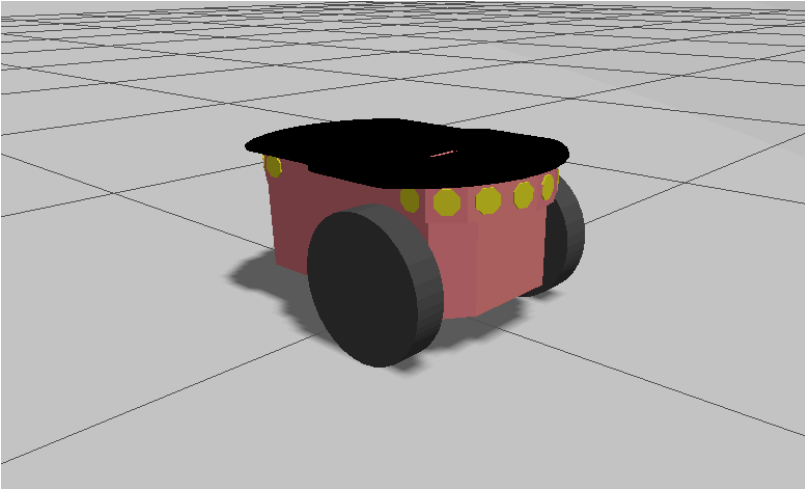
The Husky from Fuel, in our own world.

```
<left_joint>front_left_wheel_joint</left_joint>      <!-- two joints on the left... -->
<left_joint>rear_left_wheel_joint</left_joint>         <!-- ...both get one speed -->
<right_joint>front_right_wheel_joint</right_joint>     <!-- two on the right... -->
<right_joint>rear_right_wheel_joint</right_joint>      <!-- ...likewise -->
<wheel_separation>0.5708</wheel_separation>          <!-- L: left to right, m -->
<wheel_radius>0.1651</wheel_radius>                  <!-- r, m -->
```

- All left joints get one speed, all right joints another: a skid-steer, driven like a tank
- Odometry reads only the **first** joint of each side — here `front_left` and `front_right`
- There is no wheelbase tag. Every turn scrapes the tyres sideways (later)

 github.com/gazebosim/gz-sim/blob/gz-sim8/examples/worlds/diff_drive_skid.sdf

TWO WHEELS – THE PIONEER 2DX



The Pioneer 2DX from Fuel, in our own world.

```
<left_joint>left_wheel_hinge</left_joint>      <!-- one driven wheel a side -->
<right_joint>right_wheel_hinge</right_joint>    <!-- and the other -->
<wheel_separation>0.34</wheel_separation>       <!-- L, m -->
<wheel_radius>0.11</wheel_radius>              <!-- r, m -->
```

- The textbook case: two driven wheels on one axle
- Behind them, a **ball** that only carries weight. It has no friction and no joint: it just slides

 github.com/gazebo/gz-sim/blob/gz-sim8/examples/worlds/diff_drive.sdf

INSIDE THE PLUGIN

TWIST IN, WHEEL SPEEDS OUT, ODOMETRY BACK



A PLUGIN IN PYTHON

```
<!-- the loader: a system that comes with Gazebo and runs a Python module -->
<plugin filename="gz-sim-python-system-loader-system" name="gz::sim::systems::PythonSystemLoader">
  <module_name>simple_diff_drive</module_name>      <!-- our file: simple_diff_drive.py -->
  <left_joint>left_wheel_joint</left_joint>         <!-- from here on: our own tags... -->
  <right_joint>right_wheel_joint</right_joint>      <!-- ...named like the stock ones -->
  <wheel_separation>0.35</wheel_separation>        <!-- L -->
  <wheel_radius>0.1</wheel_radius>                 <!-- r -->
</plugin>
```

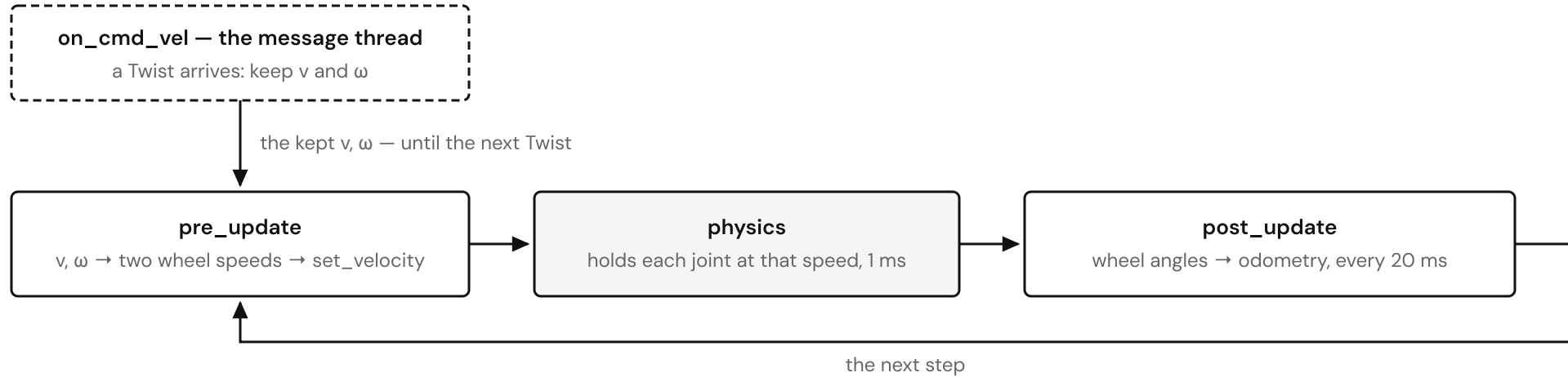
```
class SimpleDiffDrive:
    # the plugin: one object per <plugin> block
    def configure(self, entity, sdf, ecm, event_mgr): ... # once, when the world loads
    def pre_update(self, info, ecm): ...                # every step, BEFORE physics
    def post_update(self, info, ecm): ...                # every step, AFTER physics

def get_system():
    # the loader calls this once...
    return SimpleDiffDrive()                          # ...and gets the object back
```

- The loader imports the module and calls `get_system()`. Then it calls the object's methods at each moment — only the ones it has.
`configure` reads our tags by name

 gazebo-sim.org/api/sim/8/python_interfaces.html ·  github.com/gazebo-sim/gz-sim/blob/gz-sim8/examples/worlds/python_system_loader.sdf

ONE STEP, FOUR MOMENTS



- `on_cmd_vel` only **remembers** the Twist. It keeps it until the next one — there is no timeout
- `pre_update` is the only moment that may **change** the world. `post_update` only **reads** it
- The stock plugin works out the speeds after physics and applies them one step later. Ours applies them at once

ONCE, WHEN THE WORLD LOADS

```
class SimpleDiffDrive:
    def configure(self, entity, sdf, ecm, event_mgr):
        # once, when the world loads
        model = Model(entity)
        # the model this <plugin> sits in
        # <left_joint> / <right_joint>: a name in our block → a wheel joint we can command
        self.left = Joint(model.joint_by_name(ecm, sdf.get_string('left_joint')))
        self.right = Joint(model.joint_by_name(ecm, sdf.get_string('right_joint')))
        self.separation = sdf.get_double('wheel_separation')
        # L
        self.radius = sdf.get_double('wheel_radius')
        # r
        self.v = self.w = 0.0
        # no command yet: v = ω = 0
        self.node = Node()
        # Gazebo transport, not ROS
        # every Twist on /model/<name>/cmd_vel → on_cmd_vel
        self.node.subscribe(Twist, f'/model/{model.name(ecm)}/cmd_vel', self.on_cmd_vel)
```

- `Joint` is the wheel's **hinge** — the joint named in `<left_joint>`. The plugin sets its speed and reads its angle

Where these values go:

$$v_R = v + \omega \frac{L}{2}, \quad v_L = v - \omega \frac{L}{2}$$

$$\text{joint speed} = \frac{v_R}{r}, \quad \frac{v_L}{r}$$

- L and r : read **once**, used **every step**
- v and ω start at 0. They change only when a `Twist` arrives

LEGEND

joint speed — how fast a wheel turns on its hinge, rad/s

v_R, v_L — speed of each tyre, m/s

v, ω — from the `Twist`

L — distance between the wheels, m

r — wheel radius, m

FROM A TWIST TO TWO WHEEL SPEEDS

$$v_R = v + \omega \frac{L}{2}, \quad v_L = v - \omega \frac{L}{2}, \quad \text{joint speed} = \frac{v_{R,L}}{r} \text{ [rad/s]}$$

- **Which joint:** the wheel's hinge — `<right_joint>` or `<left_joint>`. Its speed is how fast that wheel spins
- When the robot turns, the outer wheel goes $\omega L/2$ faster, the inner one $\omega L/2$ slower
- Divide by r : tyre speed in m/s becomes wheel spin in rad/s. The stock DiffDrive computes the same

LEGEND

v, ω — from the `Twist`

L — distance between the wheels, m

r — wheel radius, m

v_R, v_L — speed of the right / left tyre over the floor, m/s

joint speed — how fast the wheel turns on its hinge, rad/s

```
def on_cmd_vel(self, msg):
    self.v, self.w = msg.linear.x, msg.angular.z
    # message thread, whenever a Twist arrives
    # keep v and ω, touch nothing else

def pre_update(self, info, ecm):
    if info.paused:
        return
    # every step, BEFORE physics
    # paused world: do nothing

    half = self.w * self.separation / 2
    # ω·L/2

    # set_velocity writes a JointVelocityCmd; physics then holds the joint
    # at exactly that speed for this step, whatever torque it takes
    self.left.set_velocity(ecm, [(self.v - half) / self.radius])
    self.right.set_velocity(ecm, [(self.v + half) / self.radius])
    # (v - ωL/2) / r
    # (v + ωL/2) / r
```

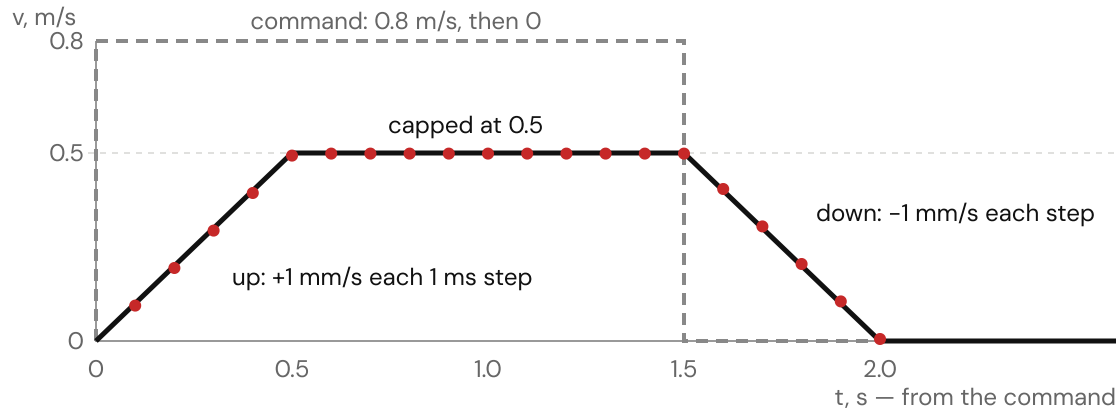
 github.com/gazebo-sim/gz-sim/blob/gz-sim8/src/systems/diff_drive/DiffDrive.cc



THE LIMITER – HOW IT WORKS

A filter between the `Twist` and the wheels: the speed may change only so fast, and never past a maximum. A real robot cannot reach full speed in 1 ms — it would slip or tip

$$a_{min} \leq \frac{v - v_{prev}}{\Delta t} \leq a_{max}, \quad v_{min} \leq v \leq v_{max}$$



LEGEND

v — the speed sent to the wheels this step
 v_{prev} — the speed of the step before
 Δt — one step, 1 ms
 a_{min}, a_{max} — slowest and fastest change, m/s² (here -1, 1)
 v_{min}, v_{max} — lowest and highest speed, m/s (here -0.5, 0.5)
The same for ω , with its own bounds

- **Yes, step by step:** each 1 ms step the speed grows by at most $a_{max} \Delta t = 1 \text{ mm/s}$ — 0 to 0.5 m/s takes 500 steps. Asked 0.8, it gets 0.5. Red dots: measured on the Pioneer, every 0.1 s

 github.com/gazebo/gz-math/blob/gz-math7/src/SpeedLimiter.cc

THE LIMITER – HOW IT IS SET

```
<plugin filename="gz-sim-diff-drive-system" name="gz::sim::systems::DiffDrive">  <!-- the same block -->
...
  <!-- joints, L, r as before -->
  <max_linear_velocity>0.5</max_linear_velocity>      <!-- v_max, m/s -->
  <min_linear_velocity>-0.5</min_linear_velocity>      <!-- v_min: reversing -->
  <max_linear_acceleration>1</max_linear_acceleration> <!-- a_max, m/s² -->
  <min_linear_acceleration>-1</min_linear_acceleration> <!-- a_min: braking -->
  <max_angular_velocity>1</max_angular_velocity>      <!-- the same for ω, rad/s... -->
  <min_angular_velocity>-1</min_angular_velocity>
  <max_angular_acceleration>2</max_angular_acceleration> <!-- ...and rad/s² -->
  <min_angular_acceleration>-2</min_angular_acceleration>
</plugin>
```

- Tags in the same `<plugin>` block. The values are from Gazebo's own example
- **No tag, no limit:** every `min_` is $-\infty$ and every `max_` is $+\infty$ until you set it. The robot can then reach any speed in one step
- Each bound is separate: `max_` alone leaves reversing unlimited. Set `min_` too

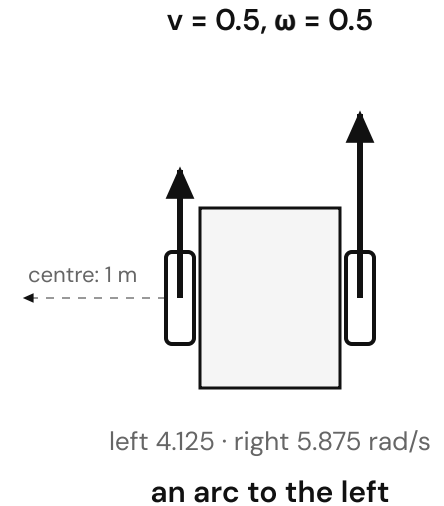
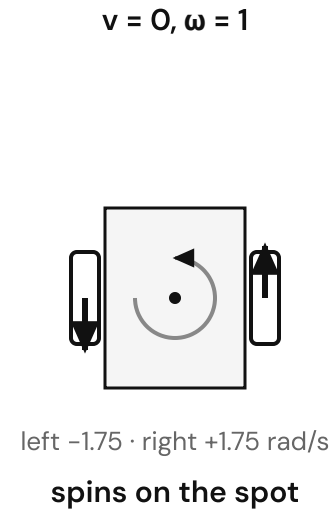
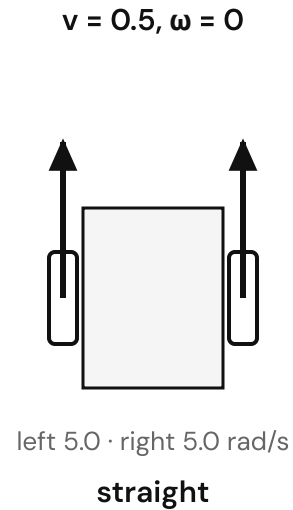
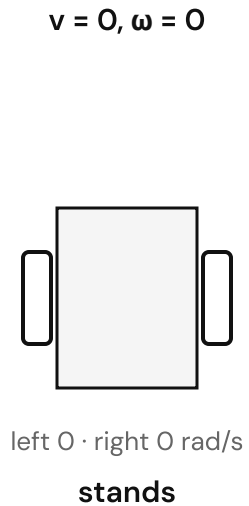
 github.com/gazebosim/gz-sim/blob/gz-sim8/examples/worlds/diff_drive_skid.sdf ·  github.com/gazebosim/gz-sim/blob/gz-sim8/src/systems/diff_drive/DiffDrive.hh

STRAIGHT, SPIN, ARC, CIRCLE

WHAT EACH WHEEL DOES



FOUR COMMANDS, TWO WHEELS

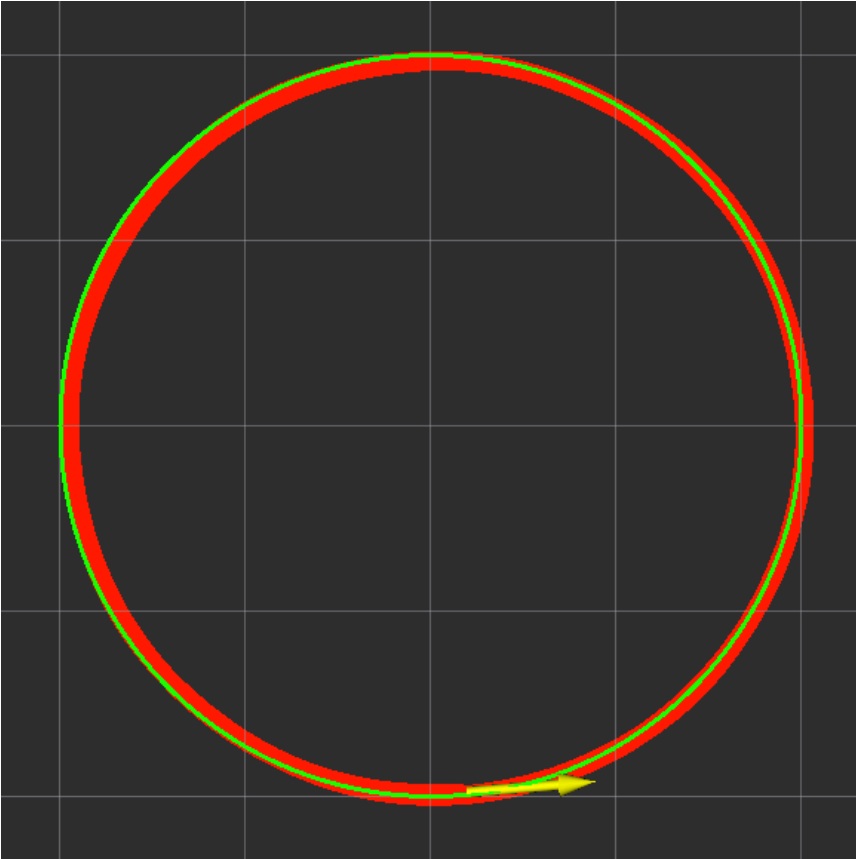


- Seen from above, forward is up. Arrow length is the wheel's speed. Positive ω turns **left** — anticlockwise, as everywhere in ROS
- $v = \omega L/2$: the inner wheel stands still, and the robot turns around it. $v < \omega L/2$: the turn centre is between the wheels, and the inner wheel runs **backwards**

LEGEND

Our robot: $L = 0.35$ m between the wheels, $r = 0.1$ m wheel radius
 Wheel speeds: joint speeds, rad/s

ONE COMMAND, A CIRCLE



Two laps, one Twist. Green: the circle commanded. Red: Gazebo's own pose.

```
# one Twist: v = 0.5 m/s, ω = 0.5 rad/s, sent once
$ ros2 topic pub --once /cmd_vel geometry_msgs/msg/Twist \
  '{linear: {x: 0.5}, angular: {z: 0.5}}' # v → linear.x, ω → angular.z
```

$$R = \frac{v}{\omega}, \quad T_{\text{lap}} = \frac{2\pi}{\omega}$$

- **One message is enough.** Nothing times out, so it circles until told otherwise
- **Green** — the circle the command asks for · **red** — where Gazebo says the robot is, two laps · **yellow arrow** — the robot now

LEGEND

R — radius of the circle, m

T_{lap} — time of one lap, s

v, ω — from the `Twist`

ODOMETRY

HOW THE WHEELS WORK OUT WHERE THE ROBOT IS



HOW ODOMETRY IS CALCULATED

Where the robot is, worked out from how far its wheels turned. Four steps, in `post_update`, every physics step:

STEP	FORMULA	IN WORDS
1 · each wheel rolled	$\Delta s_L = r \Delta \varphi_L, \quad \Delta s_R = r \Delta \varphi_R$	change of wheel angle × radius = distance on the floor
2 · the body moved	$\Delta s = (\Delta s_R + \Delta s_L) / 2$ $\Delta \theta = (\Delta s_R - \Delta s_L) / L$	average → forward; difference → turn
3 · add to the pose	$x += \Delta s \cos(\theta + \frac{\Delta \theta}{2})$ $y += \Delta s \sin(\theta + \frac{\Delta \theta}{2})$ $\theta += \Delta \theta$	one step along the heading
4 · publish	$v = \Delta s / \Delta t, \quad \omega = \Delta \theta / \Delta t, \quad q$	every 20 ms: where, which way, how fast

- The unicycle formulas, run backwards: wheels → body. Nothing outside the robot is used — a slipping wheel is invisible to it

LEGEND

φ_L, φ_R — wheel angles, as the joints report them, rad
 Δ — the change in one step
 r — wheel radius, m
 L — distance between the wheels, m
 $\Delta s_L, \Delta s_R$ — how far each tyre rolled, m
 Δs — how far the centre moved, m
 $\Delta \theta$ — how much the robot turned, rad
 x, y, θ — the pose
 Δt — one step, s
 q — the heading as a quaternion

 github.com/gazebosim/gz-math/blob/gz-math7/src/DiffDriveOdometry.cc



STEPS 1-2: WHEELS → BODY

$$\Delta s_{L,R} = r \Delta \varphi_{L,R}, \quad \Delta s = \frac{\Delta s_R + \Delta s_L}{2}, \quad \Delta \theta = \frac{\Delta s_R - \Delta s_L}{L}$$

- Each tyre rolled its wheel's angle change times the radius
- The centre moved by the **average**. The robot turned by the **difference**, divided by L

```
def post_update(self, info, ecm):  
    left = self.left.position(ecm)  
    right = self.right.position(ecm)  
    if info.paused or left is None or right is None:  
        return  
    ds_l = self.radius * (left[0] - self.last[0])  
    ds_r = self.radius * (right[0] - self.last[1])  
    self.last = (left[0], right[0])  
    ds = (ds_r + ds_l) / 2  
    dth = (ds_r - ds_l) / self.separation
```

every step, AFTER physics: read only
φ_L , rad – a list; None until asked for
φ_R , rad
nothing to read yet
$\Delta s_L = r \cdot \Delta \varphi_L$
$\Delta s_R = r \cdot \Delta \varphi_R$
φ now → "previous" next step
Δs
$\Delta \theta$

LEGEND

φ_L, φ_R – wheel angles, rad
 $\Delta \varphi$ – their change in one step
 r – wheel radius, m
 L – distance between the wheels, m
 $\Delta s_L, \Delta s_R$ – how far each tyre rolled, m
 Δs – how far the centre moved, m
 $\Delta \theta$ – how much the robot turned, rad

- Ask for the angles once, in `configure: enable_position_check(ecm)`

 gazebo.org/api/sim/8/python_interfaces.html



STEP 3: ADD IT UP

$$x += \Delta s \cos\left(\theta + \frac{\Delta\theta}{2}\right), \quad y += \Delta s \sin\left(\theta + \frac{\Delta\theta}{2}\right), \quad \theta += \Delta\theta$$

- Move Δs forward along the heading, then turn by $\Delta\theta$
- **Why $\Delta\theta/2$:** during the step the heading turns from θ to $\theta + \Delta\theta$. Stepping along the heading **halfway** through that turn follows the arc better than the start heading would

```
self.x += ds * math.cos(self.th + dth / 2)      # x += Δs·cos(θ + Δθ/2): heading at mid-step
self.y += ds * math.sin(self.th + dth / 2)      # y += Δs·sin(θ + Δθ/2)
self.th += dth                                  # θ += Δθ
```

- Odometry is a sum: every small error is added, and never taken back

LEGEND

x, y – where the robot is, m
 θ – where it faces, rad
 Δs – how far the centre moved this step, m
 $\Delta\theta$ – how much it turned this step, rad
 $+=$ – "add to what is there"

STEP 4: PUBLISH – THE ODOMETRY MESSAGE

$$v = \frac{\Delta s}{\Delta t}, \quad \omega = \frac{\Delta \theta}{\Delta t}, \quad q = \left(0, 0, \sin \frac{\theta}{2}, \cos \frac{\theta}{2}\right)$$

- **Why $\theta/2$:** a quaternion stores **half** the angle. A turn by θ about the vertical axis is $(0, 0, \sin \frac{\theta}{2}, \cos \frac{\theta}{2})$

```
if info.sim_time >= self.next_pub:           # in post_update: every 20 ms of sim time
    self.next_pub += timedelta(milliseconds=20) # the next message's time
    msg = Odometry()                          # gz.msgs.Odometry
    t = info.sim_time                         # stamp: sim time, not the wall clock...
    msg.header.stamp.sec = t.days * 86400 + t.seconds # ...whole seconds
    msg.header.stamp.nsec = t.microseconds * 1000     # ...and nanoseconds
    msg.pose.position.x, msg.pose.position.y = self.x, self.y # x, y, m
    msg.pose.orientation.z = math.sin(self.th / 2) # q: half the angle...
    msg.pose.orientation.w = math.cos(self.th / 2) # ...about z only: a flat robot
    msg.twist.linear.x = ds / info.dt.total_seconds() # v = Δs/Δt, m/s
    msg.twist.angular.z = dth / info.dt.total_seconds() # ω = Δθ/Δt, rad/s
    self.odom.publish(msg)                          # out on /model<name>/odometry
```

- Everything else stays 0 — a flat robot. The header also names the frames: `<model>/odom`,
`<model>/chassis`

LEGEND

v — forward speed, m/s

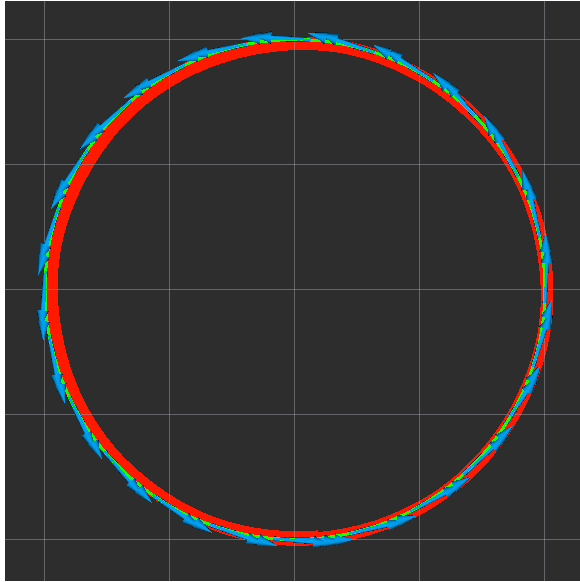
ω — turn rate, rad/s

Δt — one step, s

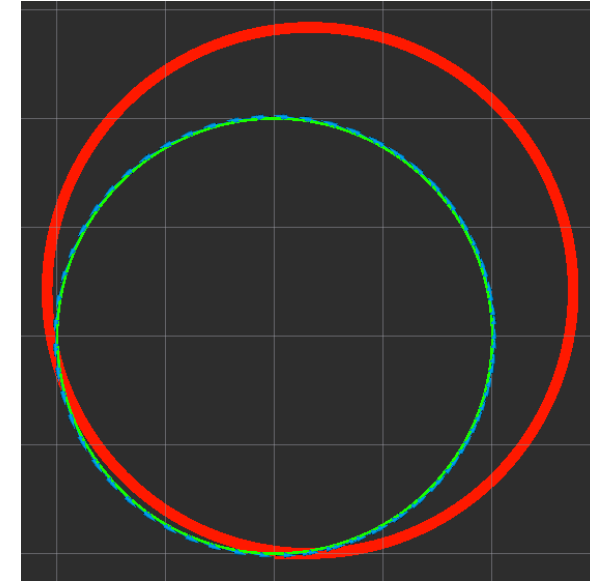
q — the heading θ as a quaternion (x, y, z, w)

```
self.odom — made once in
configure :
node.advertise(topic,
Odometry)
```


ODOMETRY VS GAZEBO: TWO WHEELS AND A HUSKY



Our two-wheeled robot, two laps. One blue arrow every 25 cm.



The Husky from Fuel, the same Twist, two laps.

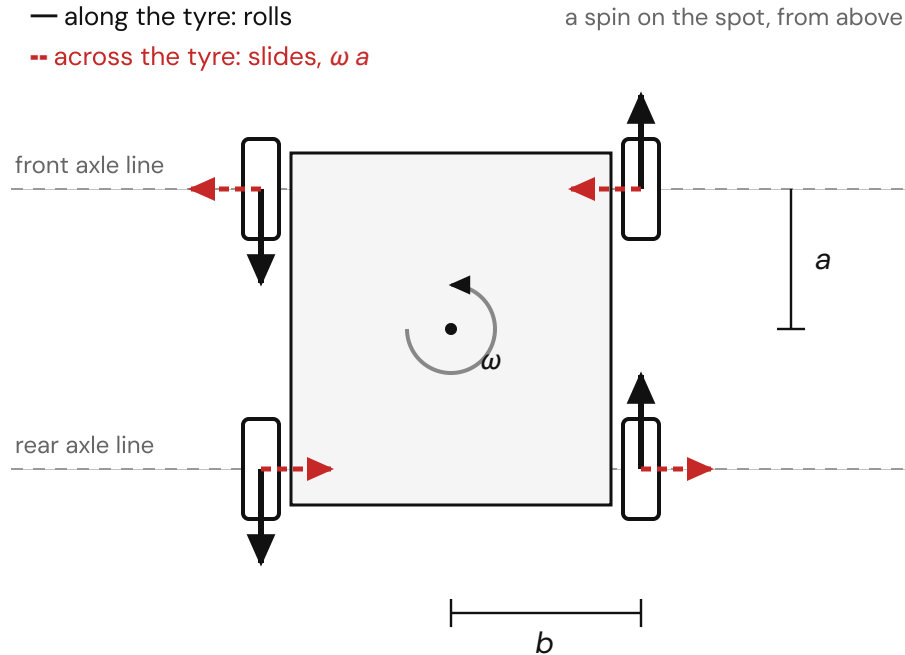
- **Green** — the circle the command asks for, $R = v/\omega \cdot \text{red}$ — where Gazebo says the robot really is • **blue arrows** — odometry: one arrow per message, where it thinks the robot is, pointing where it thinks it faces
- Blue sits on green in **both**: the wheels turn exactly as commanded, and odometry only adds up wheel angles
- **The gap between blue and red is the slip.** Two wheels: a few centimetres after two laps. The Husky: a much wider circle — after 4 s odometry says 115°, Gazebo 94°

WHY FOUR WHEELS LIE

SKID-STEER, IN FORMULAS



FOUR WHEELS CANNOT ALL ROLL THROUGH A TURN



LEGEND

a — how far the wheel's axle is ahead of (or behind) the centre: half the wheelbase, m

b — how far the wheel is to the side of the centre: half the track, m

v, ω — the robot's forward speed and turn rate

- A rolling wheel moves only **along its tyre**. So it can turn only about a point **on its axle line**
- **Two wheels, one axle:** one axle line $\rightarrow$ the turn centre lies on it $\rightarrow$ both roll
- **Four wheels, two parallel axles:** the lines never meet $\rightarrow$ no centre lets all four roll $\rightarrow$ the tyres **slide sideways**

A wheel a ahead of the centre and b to its side must move

$$v_{\text{along}}^{\text{left}} = v - \omega b, \quad v_{\text{along}}^{\text{right}} = v + \omega b, \quad v_{\text{across}} = \omega a$$

- **Any turn, not only a spin:** the sideways part ωa does not depend on v . Whenever $\omega \neq 0$, the four tyres slide
- Only $a = 0$ has nothing to slide. Odometry assumes $a = 0$ and reads the **first** joint each side: the Husky's front wheels

THE HUSKY TURNS LESS THAN IT THINKS

$$\Delta\theta_{\text{odom}} = \frac{\Delta s_R - \Delta s_L}{L} \quad \text{but} \quad \Delta\theta_{\text{true}} = \frac{\Delta s_R - \Delta s_L}{L_{\text{eff}}}, \quad L_{\text{eff}} = L \frac{\theta_{\text{odom}}}{\theta_{\text{true}}} > L$$

- The same wheel motion on top of both. The body turns less — as if the wheels were further apart
- Our Husky in Gazebo: $L_{\text{eff}} \approx 1.2 L$
- Clearpath's own ROS 2 configuration for the real Husky multiplies the distance for this reason:
`wheel_separation_multiplier: 1.875`
- What to do: measure L_{eff} (spins on the spot against a reference) · model the slip (`WheelSlip`) · or fuse an IMU — later in the course

LEGEND

$\Delta\theta_{\text{odom}}$ — the turn odometry reports, rad
 $\Delta\theta_{\text{true}}$ — the real turn, from Gazebo, rad
 $\Delta s_L, \Delta s_R$ — how far each side's tyres rolled, m
 L — distance between the wheels, from the SDF, m
 L_{eff} — the distance that would explain the real turn, m

 github.com/clearpathrobotics/clearpath_common/blob/humble/clearpath_control/config/a200/control.yaml



WILL IT TURN AT ALL?

Friction coefficient — a property of a **surface**. Each wheel sets it on its `<collision>` (the tyre); the ground has its own; physics uses the smaller of the two. The largest force a tyre can push with = coefficient × load

$$\underbrace{\text{coefficient along} \times \frac{\text{track}}{2}}_{\text{turns the robot}} > \underbrace{\text{coefficient across} \times \frac{\text{wheelbase}}{2}}_{\text{holds it still}}$$

- Wheels push forward and back → that torque **turns** it. Tyres resist sliding sideways → that torque **holds** it

```
<friction><ode>          <!-- in each wheel's <collision>; Gazebo's skid example -->
  <fdir1>0 0 1</fdir1>    <!-- direction 1 = the wheel's own z = its axle = SIDEWAYS -->
  <mu>1</mu>              <!-- friction coefficient in direction 1: across the tyre -->
  <mu2>1</mu2>            <!-- friction coefficient in direction 2: along the tyre -->
  <slip1>0.035</slip1>    <!-- sideways softness; 0 = hard -->
</ode></friction>
```

- With `slip1 = 0` it does not turn: the coefficient is 1 both ways, and the track (1.25 m) is shorter than the wheelbase (1.51 m). Holding wins: the wheels spin, the body stays
- With `slip1 = 0.035` the tyres give sideways, and it turns. The Husky: `mu` = 0.5 sideways, a track wider than its wheelbase — it turns, sliding

LEGEND

track — distance between left and right wheels, m
wheelbase — distance between front and rear axles, m
coefficient along / across — friction of the tyre along it / across it
load — the weight on the wheel; the same on both sides, so it cancels

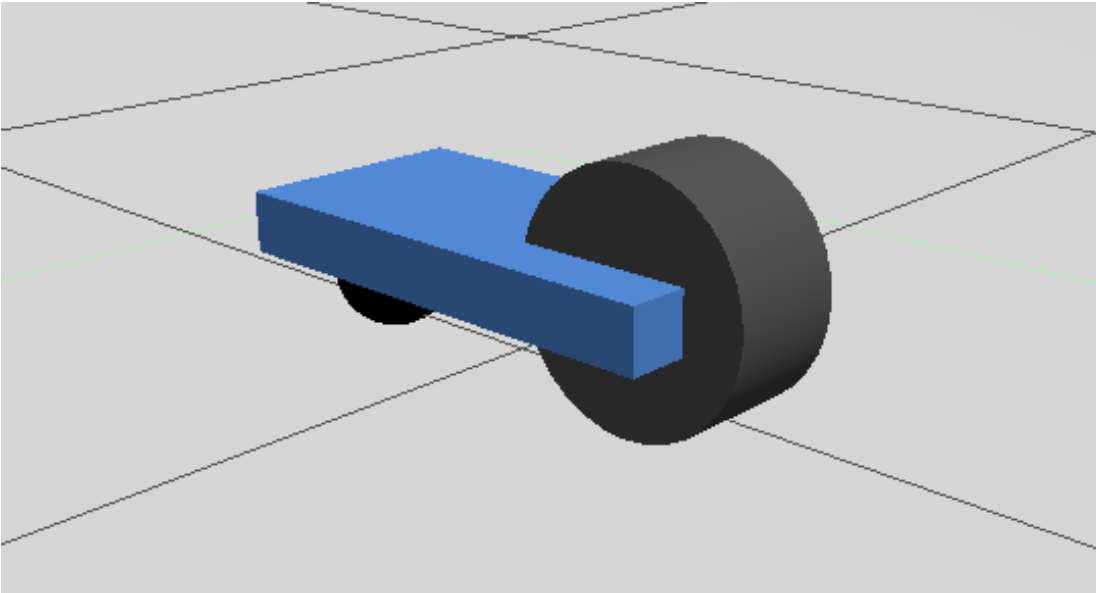
 github.com/gazebosim/gz-sim/blob/gz-sim8/examples/worlds/diff_drive_skid.sdf ·  sdformat.org/spec?ver=1.11&elem=collision

A ROBOT OF YOUR OWN

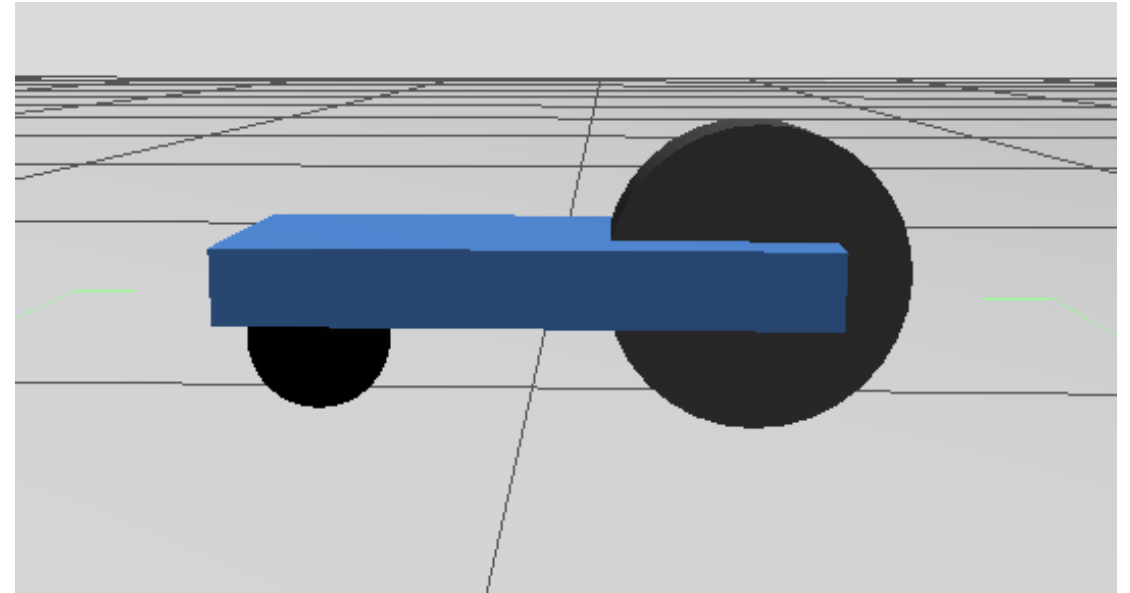
ONE WHEEL, OUR PLUGIN



ONE WHEEL – THE SIMPLEST ROBOT THAT STANDS



one_wheel, from the front right.



The same robot, side on.

- A wheelbarrow: a plate, **one wheel** in front, a **ball** behind
- The ball is the **second support point**. One wheel alone falls over. With wheel and ball, the centre of mass sits between them
- The ball has no friction, so it slides and needs no joint. No Fuel, no mesh: one world file

▶ gazebo.org/docs/harmonic/building_robot

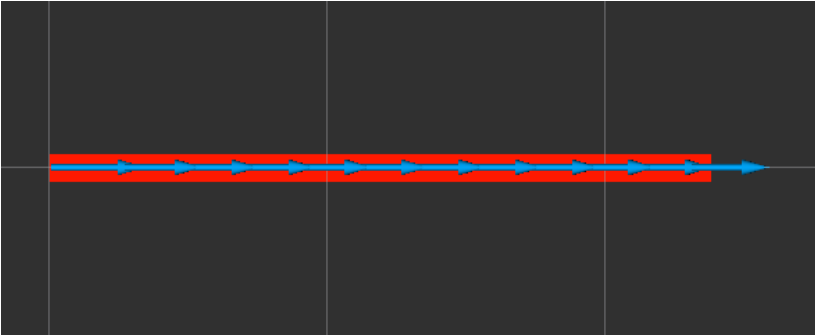
THE WHOLE ROBOT, IN MINIMAL SDF

```
<link name="body">                                <!-- the plate -->
  <pose>0 0 0.1 0 0 0</pose>                        <!-- 0.1 m above the ground -->
  <inertial auto="true"/>                            <!-- mass and inertia: from the shapes -->
  <collision name="plate"><geometry><box><size>0.4 0.2 0.05</size></box></geometry></collision>
  <collision name="ball">                            <!-- the 2nd support: without it, it tips -->
    <pose>-0.15 0 -0.05 0 0 0</pose>                <!-- behind, under the plate -->
    <geometry><sphere><radius>0.05</radius></sphere></geometry>
    <surface><friction><ode><mu>0</mu><mu2>0</mu2></ode></friction></surface> <!-- no friction: slides -->
  </collision>
</link>
<link name="wheel">
  <pose>0.15 0 0.1 -1.5708 0 0</pose>                <!-- in front; a cylinder stands on end: roll it -90° -->
  <inertial auto="true"/>                            <!-- the same: from the shape -->
  <collision name="tire"><geometry><cylinder><radius>0.1</radius><length>0.1</length></cylinder></geometry></collision>
</link>
<joint name="wheel_joint" type="revolute">           <!-- a hinge: one rotation -->
  <parent>body</parent> <child>wheel</child>        <!-- the wheel turns on the body -->
  <axis><xyz expressed_in="__model__">0 1 0</xyz></axis> <!-- the AXLE: the model's y -->
</joint>
```

- **Axle** — the line a wheel turns about: here the model's *y*, to the left. `expressed_in="__model__"` says so, so nobody has to turn the cylinder in their head
- `auto="true"` (SDF 1.11) — Gazebo computes mass and inertia from the collision shapes
- Left out here: one `<visual>` per link — the same shapes, with a colour. That is what you see

sdformat.org/spec?ver=1.11&elem=link · sdformat.org/spec?ver=1.11&elem=joint

OUR PLUGIN ON IT



"Turn left", (O, +1), for 8 s. Red: Gazebo. Blue: our odometry.

```
<!-- the same Python loader and module as before -->
<plugin filename="gz-sim-python-system-loader-system" name="gz::sim::systems::PythonSystemLoader">
  <module_name>simple_diff_drive</module_name>    <!-- our SimpleDiffDrive -->
  <left_joint>wheel_joint</left_joint>           <!-- the one wheel... -->
  <right_joint>wheel_joint</right_joint>          <!-- ...named on both sides -->
  <wheel_separation>0.3</wheel_separation>        <!-- L: no 2nd wheel, the formula wants one -->
  <wheel_radius>0.1</wheel_radius>               <!-- r -->
</plugin>
```

- `teleop_twist_keyboard` drives it as it is. The bridge carries `/cmd_vel` to the model's topic
- "Turn left" drives it **forward**, straight — and the odometry agrees
- The stock DiffDrive does exactly the same on this robot

**ONE WHEEL, AND "TURN LEFT" DRIVES IT FORWARD.
WHY?**



THE LAST WRITE WINS

- `pre_update` writes the left speed into the joint, then the right speed — **into the same joint**, `wheel_joint`. Physics sees only the second:

$$\text{joint speed} = \frac{v + \omega L/2}{r}$$

- For example $v = 0, \omega = 1, L/2 = 0.15$: the wheel turns forward, the robot goes 0.15 m/s ahead. The code is right; the robot is wrong
- Odometry reads that joint twice: $\Delta s_R = \Delta s_L$, so $\Delta \theta = 0$ — "straight". And it is right
- One wheel on the centre line pushes only along it. The ball gives no side force. To turn you need **two** wheels — or one **steered** wheel: `AckermannSteering`
- With no `right_joint`: nothing moves, no odometry, no warning

LEGEND

joint speed — how fast
`wheel_joint`, the wheel's hinge,
turns, rad/s
 v, ω — from the `Twist`
 L — `wheel_separation`, 0.3 m:
there is no second wheel, but
the formula needs it
 r — wheel radius, 0.1 m

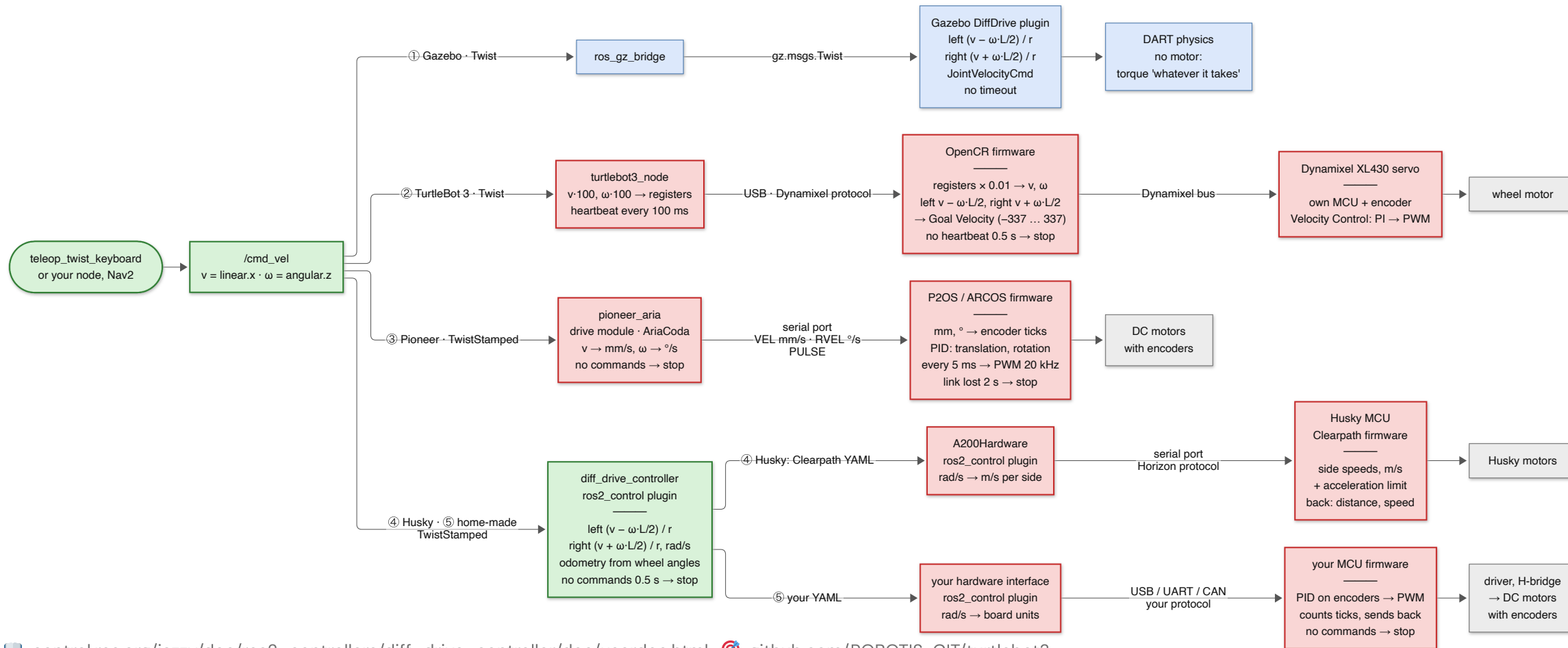
FROM SIMULATION TO HARDWARE

THE PLUGIN STAYS BEHIND



ONE `cmd_vel`, FIVE PATHS TO A WHEEL

■ shared by all or several robots · ■ specific to one robot — exactly what changes when you move · ■ simulation only · ■ hardware



control.ros.org/jazzy/doc/ros2_controllers/diff_drive_controller/doc/userdoc.html · github.com/ROBOTIS-GIT/turtlebot3

OUR ROBOTS, SIMULATED AND REAL

	① GAZEBO	② TURTLEBOT 3	③ PIONEER	④ HUSKY · ⑤ HOME-MADE
the message you send	Twist	Twist	TwistStamped	TwistStamped
who turns (v, ω) into wheel speeds	the plugin	OpenCR firmware	ARCOS firmware	diff_drive_controller
what replaces <code>set_velocity</code>	— physics holds the joint	a speed command to each Dynamixel servo	ARCOS drives the motors	the Husky's MCU · your firmware
commands stop coming	drives on	drives on — until its node dies	stops	stops after 0.5 s

- **Shared by all five: the meaning.** $v = \text{linear.x}$, $\omega = \text{angular.z}$, in m/s and rad/s, in the robot's own axes
- **The formulas are everywhere too:** left $(v - \omega L/2)/r$, right $(v + \omega L/2)/r$ — in a plugin, in two firmwares and in a controller. Only their address changes
- **Per robot: what replaces `set_velocity`** — the driver, the firmware and the wire between them. That is the red, and that is what you replace when you move

[▶ emanual.robotis.com/docs/en/platform/turtlebot3/overview](https://emanual.robotis.com/docs/en/platform/turtlebot3/overview) ·
 [🔗 github.com/grupo-avispa/pioneer_ros2](https://github.com/grupo-avispa/pioneer_ros2) ·
 [🔗 github.com/clearpathrobotics/clearpath_common](https://github.com/clearpathrobotics/clearpath_common)



A PERFECT MOTOR – AND A REAL ONE

- `set_velocity` is **not a force**. It tells physics "this joint turns at this speed", and physics applies whatever torque that takes. So a wheel reaches its speed in one step, and stops in one
- A force wins: if anything else sets a force on that joint, the speed command is ignored

	GAZEBO, <code>set_velocity</code>	A REAL ROBOT
who holds the speed	the physics engine — exactly, every step	firmware in a microcontroller, or inside the servo
feedback	the joint's exact speed, no noise	an encoder: ticks — rough and noisy
what turns the wheel	a torque, applied straight to the joint	PWM → H-bridge → current → motor → gearbox
limits	none, unless the model sets them	battery voltage, driver current, motor torque
what odometry sees	the ideal wheel angle	ticks — with slip and backlash

- On a real robot the speed loop lives **in firmware**, not in ROS 2. It must also stop the motors **by itself** when the computer hangs

gazeboapi.org/api/sim/8/classgz_1_1sim_1_1Joint.html · github.com/joshnewans/ros_arduino_bridge · articulatedrobotics.xyz/tutorials/mobile-robot/applications/ros2_control-concepts

DOES YOUR NODE CHANGE?

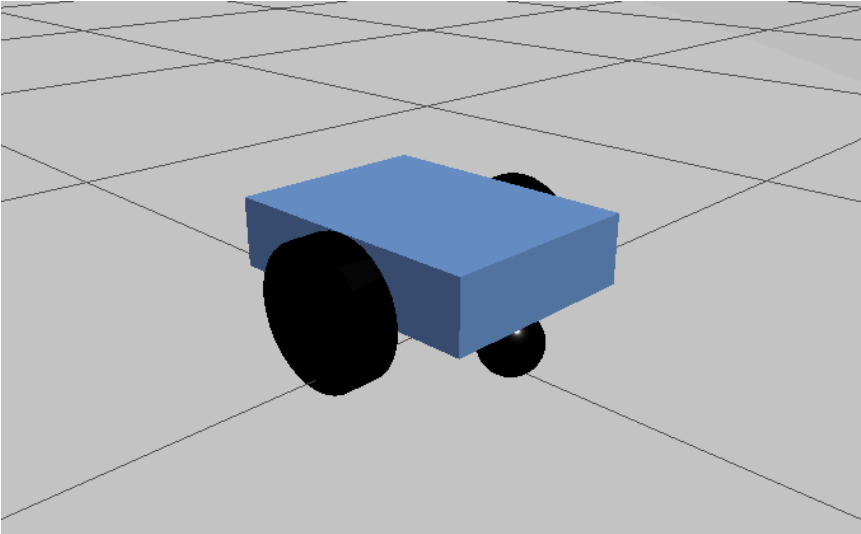
WHAT	DOES IT CHANGE?	SO THAT YOUR CODE DOES NOT
the algorithm — which v and ω to send	no	—
the message type	<code>Twist</code> in simulation and on a TurtleBot 3, <code>TwistStamped</code> elsewhere	take the type from a parameter, like teleop's <code>stamped</code>
the topic name	sometimes — <code><ns>/cmd_vel</code> on a Husky	a relative <code>cmd_vel</code> , and a remap in the launch file
how often you send	yes, if you send once: a timeout stops the robot	publish on a timer
whether it is obeyed literally	no — the robot's limits clip it	close the loop on <code>odom</code> , not on what you sent
the clock	<code>use_sim_time: false</code>	a parameter, not code

- Publish on a timer, take the type from a parameter and the topic from a remap — and on the robot **only the launch file changes**

 github.com/ros2/teleop_twist_keyboard/blob/jazzy/teleop_twist_keyboard.py



EXERCISE (OPTIONAL) – YOUR OWN DRIVE



custom_plugin_bot: two wheels, a caster, a Python plugin.

Not graded — for those who want to write a Gazebo plugin.

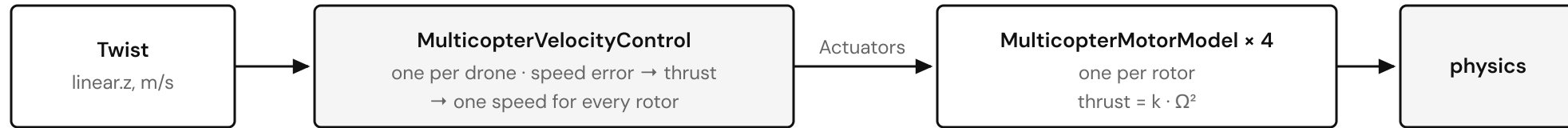
1. **Build the drive** — the loader block, `configure`, `pre_update`: slides 11, 13, 14
2. **Make it report** — `post_update` and a `gz.msgs.Odometry`: Δs , $\Delta \theta$, the pose, v , ω , q — slides 22, 23, 24
3. **Check it** — on a circle command your odometry must equal the ideal $(R \sin \omega t, R(1 - \cos \omega t), \omega t)$. Gazebo's pose will not — say why
4. **Try it on robots that already exist:**
 - `one_wheel` — does "turn left" do what slide 35 says?
 - Pioneer 2DX, Fuel — two wheels, the closest to ideal
 - Husky, Fuel — make it drive all four joints, then watch it lie
 - TurtleBot 3, Fuel — fix its wheel radius first

ROTORS

HOW A DRONE'S PLUGINS DIFFER FROM DIFFDRIVE



TWO PLUGINS, AGAIN



- The **motor** knows one rotor and nothing about the aircraft — the drone's version of the Husky's wheel joint
- The **controller** is the brain: it reads the drone's real velocity from Gazebo and decides how hard to push
- To go straight up, all four spin **equally**: the thrust stays vertical, and the twisting torques cancel — two rotors turn clockwise, two anticlockwise
- Up and down only. How the controller works, and writing a motor of our own — a later lecture

gazeboapi.org/sim/8/classgz_1_1sim_1_1systems_1_1MulticopterVelocityControl.html

A MOTOR IS NOT A DRIVE

	DiffDrive	MulticopterMotorModel + MulticopterVelocityControl
plugins per robot	one	one per rotor , and one controller
input	Twist — an intention	one speed per rotor, from the controller
what it sets every step	each wheel joint's speed — physics holds it exactly	the rotor's speed, and a force along the rotor axis with a twist on the body
formula	left $(v - \omega L/2)/r$, right $(v + \omega L/2)/r$ — exact kinematics	$T = k \Omega^2$ — then physics integrates it
who closes the loop	nobody needs to: the ground holds the robot level	the controller, watching the real velocity
odometry	yes, from the wheel angles	none — a separate plugin
on a real robot	firmware holds each wheel's speed: OpenCR, ARCOS, the Husky's MCU	an autopilot — PX4, ArduPilot — and an ESC per motor

LEGEND

v, ω — from the Twist
 L — distance between the wheels, m
 r — wheel radius, m
 T — thrust of one rotor, N
 k — motorConstant
 Ω — the rotor's spin speed, rad/s

- The wheel formula is exact: set the speeds, and the robot goes there. The rotor formula only says how hard to push — something must watch the result

 github.com/gazebo/gz-sim/blob/gz-sim8/src/systems/multicopter_motor_model/MulticopterMotorModel.cc



WHAT'S NEXT

Lecture 4 — Building the Robot

Describing a robot yourself in URDF · parts, joints and frames · `ros2_control`, the same in simulation and on hardware · the robot that finally appears in RViz

 docs.ros.org/en/jazzy/Tutorials/Intermediate/URDF/URDF-Main.html

